# 22. Courier Delivery Route & Tracking

## Introduction

This Java program demonstrates several fundamental courier-delivery operations in one console-based application. It calculates delivery charges, tracks package statuses, checks distance eligibility, searches for tracking IDs, updates package statuses, analyzes delivery distances, identifies the farthest delivery distance, and displays a courier route matrix.

Together, these operations show how basic Java input, decision-making, arrays, loops, and two-dimensional data structures can model practical delivery and tracking tasks.

---

## Objectives

- **Calculate delivery charges** based on package weight.
- **Track package statuses** using numeric status values and switch-case logic.
- **Check delivery eligibility** according to a maximum distance limit.
- **Search tracking IDs** and report whether a requested ID exists.
- **Update package statuses** at a specified position in an array.
- **Analyze delivery distances** by calculating totals, averages, and counts.
- **Find the farthest delivery** from a collection of distances.
- **Represent courier routes** with a two-dimensional matrix.
- **Practice core Java concepts** through reusable console input and readable output.

---

## Complete Java Program

The following source is reproduced exactly as provided and presented in a monospaced format.

```
import java.util.*;

public class PROJECT {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int t = sc.nextInt();
        for (int i = 0; i < t; i++) {
            int w = sc.nextInt();
            int charge = 50 + (10 * w);
            System.out.println(charge);
        }

        int t2 = sc.nextInt();
        for (int i = 0; i < t2; i++) {
            int s = sc.nextInt();

            switch (s) {
                case 1:
                    System.out.println("Package Created");
                    break;
                case 2:
                    System.out.println("In Transit");
                    break;
                case 3:
                    System.out.println("Delivered");
                    break;
                default:
                    System.out.println("Invalid Status");
            }
        }

        int t3 = sc.nextInt();
        for (int i = 0; i < t3; i++) {
            int dis = sc.nextInt();

            if (dis <= 500) {
                System.out.println("Eligible");
            } else {
                System.out.println("Not Eligible");
            }
        }

        int n = sc.nextInt();
        int[] trackingId = new int[n];
```

```
        for (int i = 0; i < n; i++) {
            trackingId[i] = sc.nextInt();
        }

        int searchingId = sc.nextInt();
        boolean found = false;

        for (int i = 0; i < n; i++) {
            if (trackingId[i] == searchingId) {
                found = true;
                break;
            }
        }

        System.out.println(found ? "Found" : "Not found");

        int n2 = sc.nextInt();
        int[] status = new int[n2];

        for (int i = 0; i < n2; i++) {
            status[i] = sc.nextInt();
        }

        int position = sc.nextInt();
        int newStatus = sc.nextInt();

        status[position - 1] = newStatus;

        for (int i = 0; i < n2; i++) {
            System.out.print(status[i] + " ");
        }
        System.out.println();

        int n3 = sc.nextInt();
        int total = 0;
        int count = 0;

        for (int i = 0; i < n3; i++) {
            int distance = sc.nextInt();
            total += distance;

            if (distance > 100) {
                count++;
            }
        }
```

```
        double average = (double) total / n3;
        System.out.println("Total=" + total);
        System.out.println("Average=" + average);
        System.out.println("Count=" + count);

        int n4 = sc.nextInt();
        int[] farDistance = new int[n4];
        int max = Integer.MIN_VALUE;

        for (int i = 0; i < n4; i++) {
            farDistance[i] = sc.nextInt();

            if (farDistance[i] > max) {
                max = farDistance[i];
            }
        }

        System.out.println(max);

        int n5 = sc.nextInt();
        int[][] route = new int[n5][n5];

        for (int i = 0; i < n5; i++) {
            for (int j = 0; j < n5; j++) {
                route[i][j] = sc.nextInt();
            }
        }

        for (int i = 0; i < n5; i++) {
            for (int j = 0; j < n5; j++) {
                System.out.print(route[i][j] + " ");
            }
            System.out.println();
        }

        sc.close();
    }
}
```

---

# Program Explanation

# Delivery Charge Calculation

For each package weight, the program calculates the charge with the formula **50 + (10 × weight)**. The fixed base charge is 50, while an additional charge of 10 is applied for every unit

of weight.

## Package Status Tracking

The switch statement interprets each status value. A value of 1 means “Package Created,” 2 means “In Transit,” and 3 means “Delivered.” Any other value produces “Invalid Status.”

## Distance Eligibility

The program checks whether each delivery distance is less than or equal to 500. Deliveries within this limit are reported as “Eligible”; distances above it are reported as “Not Eligible.”

## Tracking-ID Search

The tracking-ID array is traversed from the first element to the last. The boolean variable **found** begins as false and changes to true when the requested tracking ID is located.

## Status Update

The program subtracts 1 from the supplied position because array indexes begin at zero. It then replaces the value at that adjusted index with the new status.

## Distance Statistics

Each distance is added to **total**. The average is calculated by dividing the total by the number of distances, while **count** records how many distances are greater than 100.

## Maximum Distance

The variable **max** starts at Integer.MIN_VALUE, ensuring that the first distance will replace it even when all supplied distances are negative. Each later value is compared with the current maximum.

## Route Matrix

The two-dimensional route array stores values in rows and columns. Nested loops first read every matrix element and then display the values row by row, representing route relationships in a structured form.

---

## Advantages

- **Modular courier operations** — The program handles charging, tracking, eligibility, analysis, and route display as distinct tasks.
- **Effective use of arrays** — Arrays store tracking IDs, package statuses, and delivery distances in an organized way.

- **Efficient linear search** — The tracking-ID search stops immediately when a matching ID is found.
- **Clear decision logic** — Conditional statements and switch-case branches make business rules easy to follow.
- **Route-matrix representation** — A two-dimensional array provides a simple structure for displaying route data between locations.
- **Reusable console input** — A single Scanner object supports all required input operations throughout the program.
- **Easy extensibility** — Additional statuses, fee rules, validation checks, or route features can be added as the system grows.
- **Readable output** — Descriptive status messages and labeled statistics help users interpret the results.
- **Strong learning value** — The program is suitable for beginner-to-intermediate Java practice because it combines common language features in a practical context.

**Practical limitations.** A production courier system would need stronger input validation, exception handling, persistent data storage, and more advanced route-optimization methods. These additions would improve reliability, scalability, and real-world decision-making.

---

## Sample Input and Output

The following is an illustrative example, not the only valid input. It follows the exact order of values read by the program.

## Example Input

```
2
3 5
3
1 2 4
2
450 700
4
101 202 303 404
303
3
1 2 3
2 3
4
80 120 250 60
3
25 600 140
2
0 12
12 0
```

## Example Output

```
80
100
Package Created
In Transit
Invalid Status
Eligible
Not Eligible
Found
1 3 3
Total=510
Average=127.5
Count=2
600
0 12
12 0
```

---

## Conclusion

The program combines basic Java concepts to model essential courier delivery, route, and tracking tasks. Through console input, arithmetic calculations, switch-case logic, conditions,

arrays, linear search, statistical analysis, and nested loops, it provides a clear academic example of how programming fundamentals can support a practical delivery-management scenario.